RIPHAH INTERNATIONAL UNIVERSITY

PROJECT REPORT

LR(0) Parser

Interactive Bottom-Up Parsing Engine with DFA Visualization, Canonical Collection Construction, and Full Parse Trace

PROJECT DETAILS		
Group Members	Mohammad Sami	**53199**
Instructor	Mr. Tajamul Shazad	
Course	Compiler Construction	
Semester	6th Semester | 2026	

.

1. INTRODUCTION

LR(0) parsing is one of the simplest bottom-up parsing techniques based on the shift-reduce approach. It provides a strong foundation for understanding more advanced parsing techniques such as SLR and CLR.

This project presents a complete implementation of an LR(0) parser developed in Python, supported by a graphical user interface built using Tkinter. The system allows users to input any context-free grammar and observe each phase of parsing, including grammar augmentation, canonical collection construction, parsing table generation, and string validation.

To improve understanding, the system also includes DFA visualization and a detailed parsing trace.

.

1.1 Objectives

- Accept any context-free grammar as input
- Automatically augment the grammar
- Construct canonical LR(0) item sets
- Generate ACTION and GOTO parsing tables
- Detect conflicts in parsing
- Perform step-by-step parsing
- Visualize the DFA

.

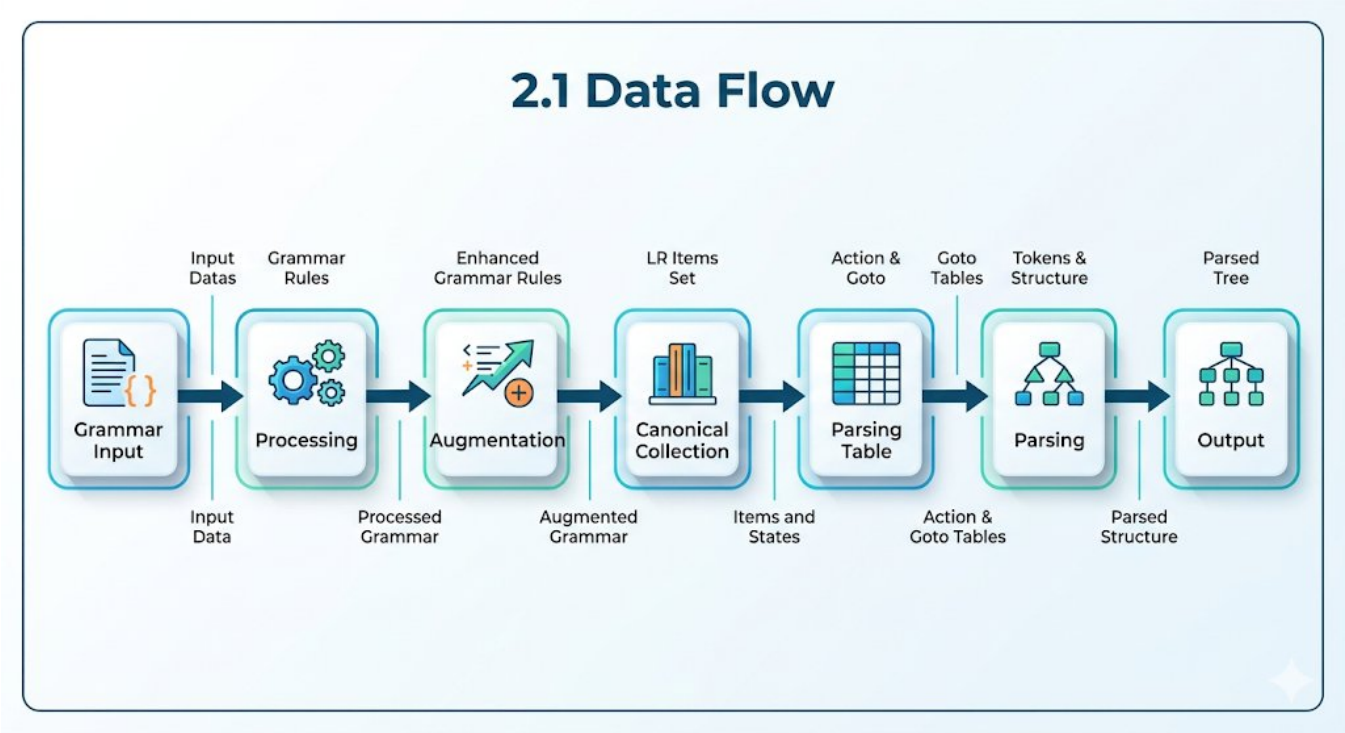
2. SYSTEM ARCHITECTURE

The system follows a modular layered architecture:

Layer	Description
Grammar Layer	Parses and augments grammar
Automata Layer	Builds LR(0) states
Table Layer	Constructs parsing tables
Parser Layer	Executes shift-reduce parsing
GUI Layer	Displays results

.

2.1 Data Flow



.

3. MODULE DESCRIPTION

3.1 Grammar Module

- Handles grammar parsing, tokenization, and augmentation.

3.2 Automata Module

- Builds LR(0) states using closure and GOTO functions.

3.3 Table & Parser Module

- Constructs parsing tables and performs parsing with trace.

3.4 GUI Module

- Displays DFA, tables, and parsing steps interactively.

.

4. TEST CASE

4.1 Input Grammar

S → A A
A → a A | b

.

4.2 Augmented Grammar

S' → S
S → A A
A → a A
A → b

.

4.3 Canonical Collection

The system generates **7 LR(0) states**, forming the DFA required for parsing.

.

5. LR(0) PARSING TABLE

Parsing Table

State	ACTION			GOTO	
	a	b	$	S	A
I0	s3	s4	—	1	2
I1	—	—	acc	—	—
I2	s3	s4	—	—	5
I3	s3	s4	—	—	6
I4	r4	r4	r4	—	—
I5	r2	r2	r2	—	—
I6	r3	r3	r3	—	—

.

Legend

- sX → Shift
- rX → Reduce
- acc → Accept
- — → Error

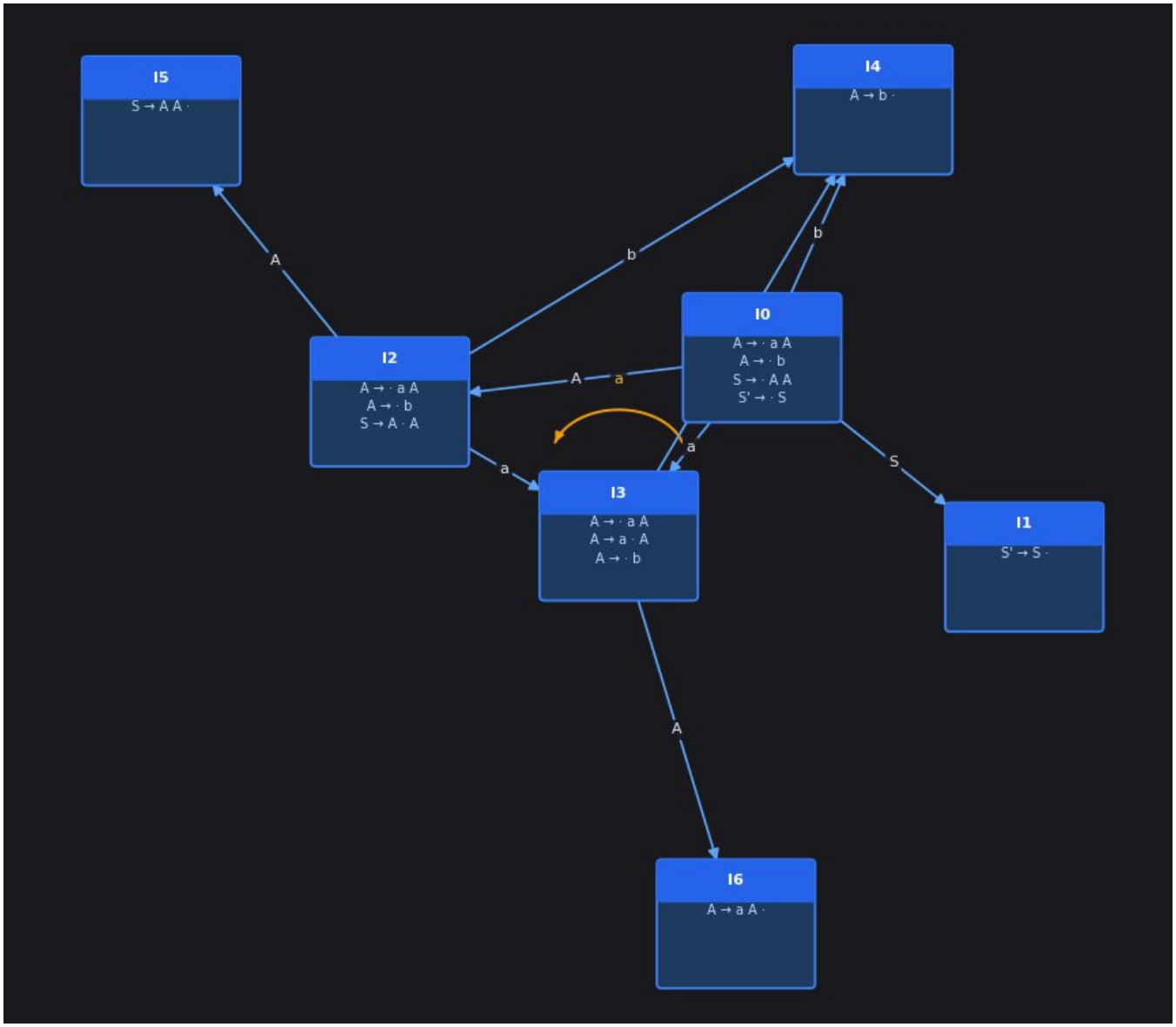
.

Production Rules

No.	Production
1	S' → S
2	S → A A
3	A → a A
4	A → b

.

5.1 DFA Diagram



.

6. PARSE TRACE

Input String:

a b b

.

Step-by-Step Parsing Table

Step	Stack	Input	Action
1	0	a b b $	Shift s3
2	0 a 3	b b $	Shift s4
3	0 a 3 b 4	b $	Reduce r4 (A → b)
4	0 a 3 A 6	b $	Reduce r3 (A → a A)
5	0 A 2	b $	Shift s4
6	0 A 2 b 4	$	Reduce r4 (A → b)
7	0 A 2 A 5	$	Reduce r2 (S → A A)
8	0 S 1	$	**ACCEPT**

.

Result

The input string **"a b b" is ACCEPTED**, confirming that it belongs to the given grammar.

.

7. CONCLUSION

This project successfully demonstrates the complete implementation of an LR(0) parser, including grammar processing, DFA construction, parsing table generation, and string validation.

The modular design ensures clarity and extensibility, while the graphical interface enhances usability. The inclusion of DFA visualization and detailed parse tracing makes the system effective for both learning and demonstration purposes.

.

8. GITHUB

https://github.com/MohmmadSami/lr0_parser.git